

Project Title: API Security Assessment of OWASP Juice Shop

Tester: M. Bhuvanesh Kumar

Tools Used:

- Burp Suite Professional
- JWT.io
- Firefox Browser
- OWASP Juice Shop

Testing Methodology:

- OWASP API Security Top 10
- Manual API Security Testing

1. Executive Summary

This assessment was conducted against the OWASP Juice Shop application to identify security weaknesses within exposed APIs.

The testing focused on authentication mechanisms, authorization controls, sensitive data exposure, and object-level access controls.

Several vulnerabilities were identified including:

- Broken Object Level Authorization (BOLA)
- Excessive Data Exposure
- Broken Authentication
- Sensitive Information Exposure

These vulnerabilities could allow attackers to access unauthorized resources, expose confidential information, and bypass authentication controls.

2. Scope

Application

OWASP Juice Shop

Target:

<http://localhost:3000>

Testing Type

Black Box API Security Testing

3. Methodology

Phase 1 – Application Reconnaissance

Objective:

Understand application functionality and identify available API endpoints.

Figure 1 – OWASP Juice Shop Home Page

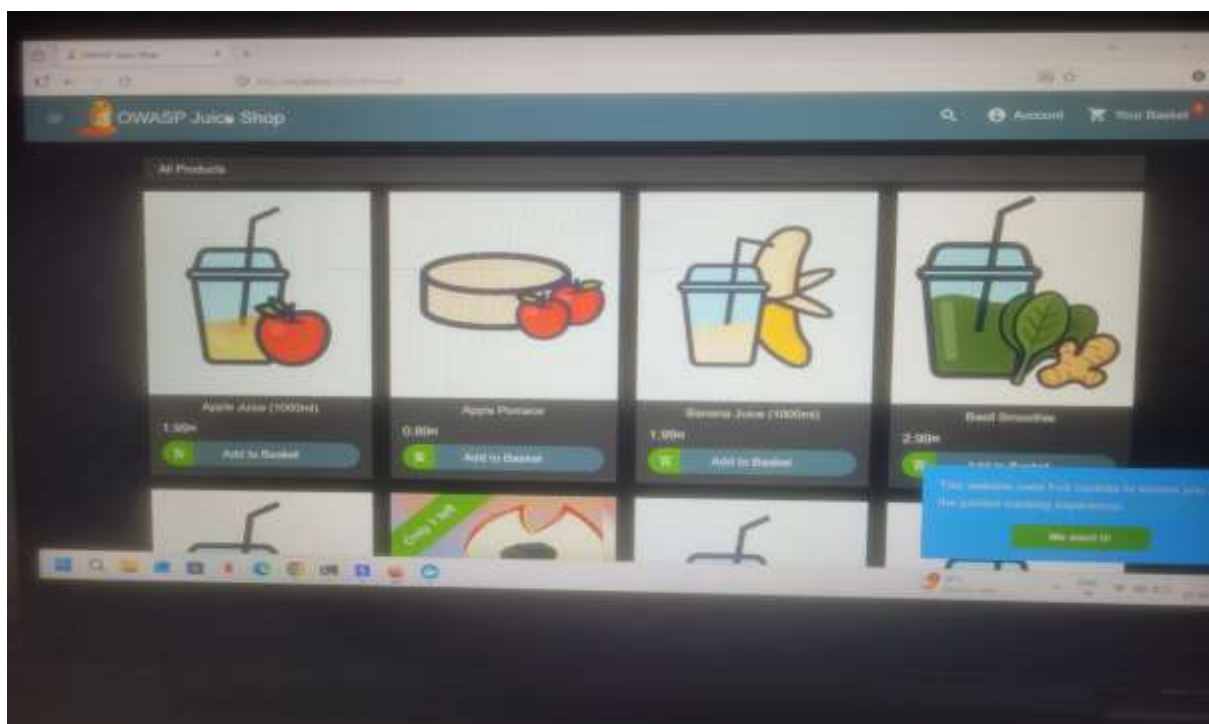
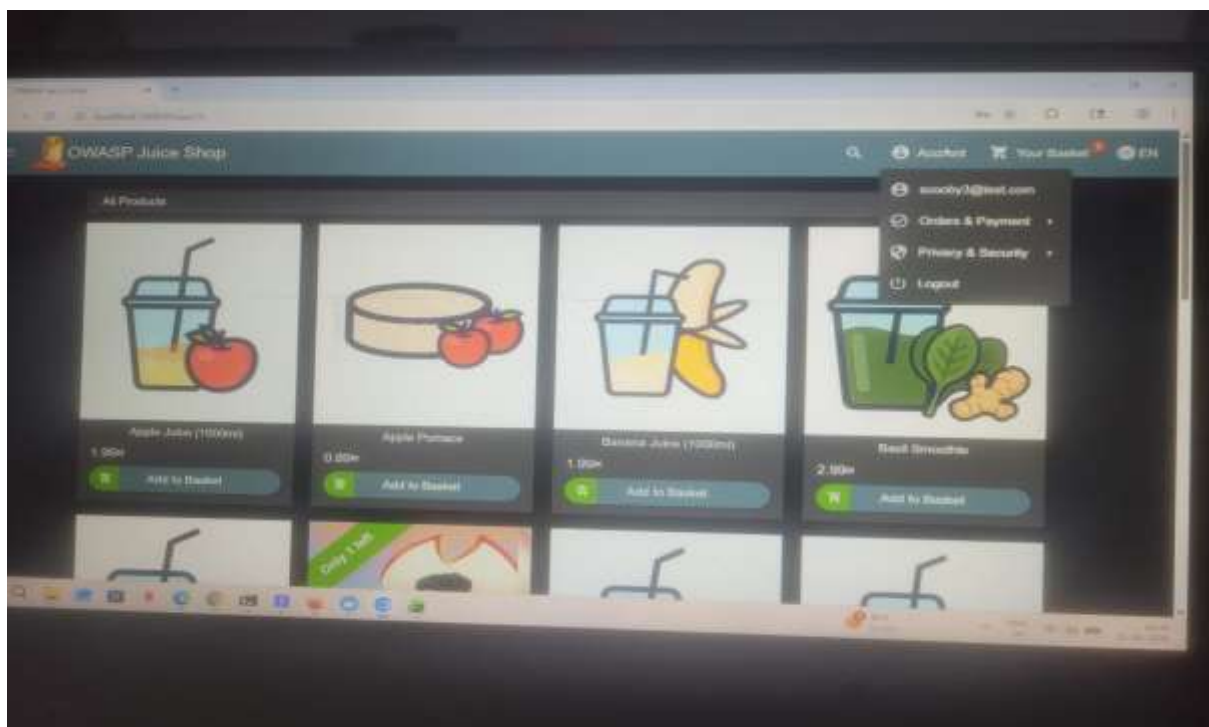


Figure 2 – Authenticated User Session



Phase 2 – API Discovery

Objective:

Identify API endpoints exposed by the application.

Burp Suite Proxy and Site Map were used to discover API endpoints.

Figure 3 – Burp Site Map Showing Discovered API Endpoints

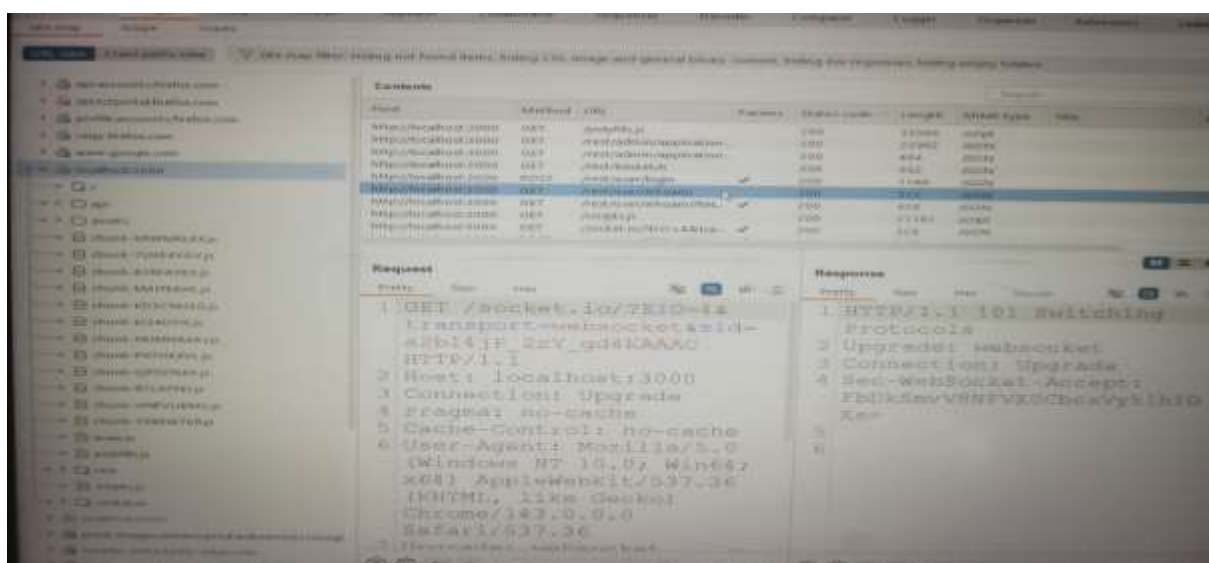


Figure 4 – API Endpoint Enumeration

Help Burp Suite Professional v2023.12.2 - Temporary Project - local

Repeater Collaborator Sequencer Decoder Comparer Logger Organizer

Hiding not found items; hiding CSS, image and general binary content; hiding 4xx responses; hiding empty folders

Contents

Host	Method	URL	Params	Status code	Length	MIME type	Title
http://localhost:3000	GET	/socket.io/?EIO=4&tra...	✓	101	129		
http://localhost:3000	GET	/socket.io/?EIO=4&tra...	✓	101	129		
http://localhost:3000	GET	/socket.io/?EIO=4&tra...	✓	101	129		
http://localhost:3000	GET	/		200	10370	HTML	OWASP
http://localhost:3000	POST	/api/BasketItems/	✓	200	542	JSON	
http://localhost:3000	GET	/api/Products/517d=Fri...	✓	200	665	JSON	
http://localhost:3000	GET	/api/Quantities/		200	8016	JSON	
http://localhost:3000	GET	/api/SecurityQuestions/		200	2321	JSON	
http://localhost:3000	GET	/chunk-6RWMKLEX.js		200	139342	script	
http://localhost:3000	GET	/chunk-7U6P4YGV.js		200	97091	script	
http://localhost:3000	GET	/chunk-B3RFAYKZ.js		200	1025	script	
http://localhost:3000	GET	/chunk-EMJTEAHL.js		200	33621	script	
http://localhost:3000	GET	/chunk-KD3CNUZG.js		200	10205	script	
http://localhost:3000	GET	/chunk-KI34UYIE.js		200	147754	script	
http://localhost:3000	GET	/chunk-PX7UKXVL.js		200	1422	script	
http://localhost:3000	GET	/chunk-R7LX7FKI.js		200	41685	script	
http://localhost:3000	GET	/chunk-UNFVL3M2.js		200	78038	script	
http://localhost:3000	GET	/chunk-V3BEW76R.js		200	1486	script	
http://localhost:3000	GET	/main.js		200	784254	script	
http://localhost:3000	GET	/polyfills.js		200	35066	script	
http://localhost:3000	GET	/rest/basket/6		200	832	JSON	
http://localhost:3000	GET	/rest/captcha/		200	431	JSON	
http://localhost:3000	GET	/rest/saveLoginIp		200	343		
http://localhost:3000	POST	/rest/user/login	✓	200	1192	JSON	
http://localhost:3000	GET	/rest/user/security-qu...	✓	200	384	JSON	
http://localhost:3000	GET	/rest/user/security-qu...	✓	200	384	JSON	
http://localhost:3000	GET	/rest/user/whoami		200	513	JSON	
http://localhost:3000	GET	/socket.io/?EIO=4&tra...	✓	200	326	JSON	

Request

Pretty Raw Hex

1 GET / HTTP/1.1

1 HTTP/1.1 200 OK

0 highlights

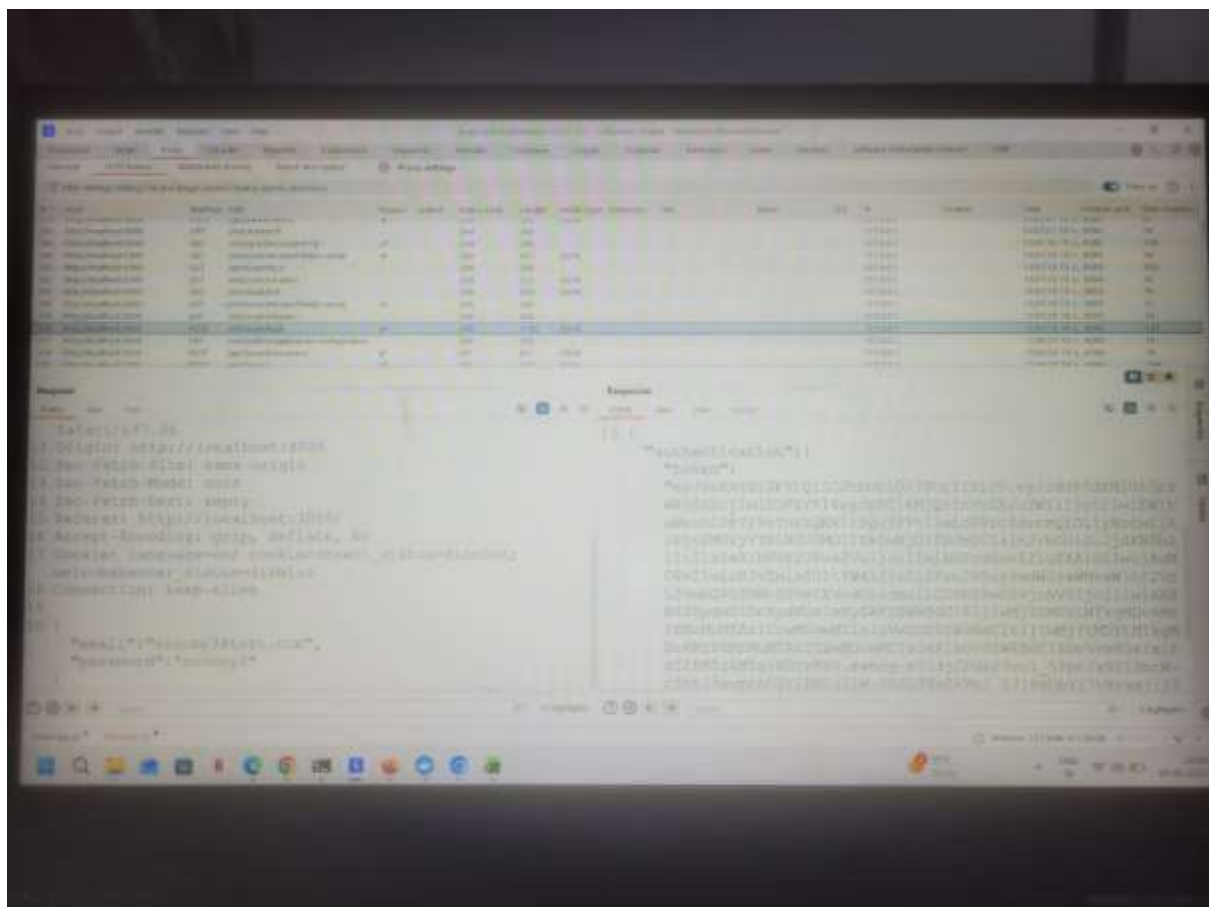
Phase 3 – Authentication Analysis

Objective:

Analyze login process and JWT implementation.

The login request was intercepted using Burp Suite and the JWT token was extracted for analysis.

Figure 5 – Successful Login Request Returning JWT Token



Phase 4 – JWT Analysis

Objective:

Decode and analyze JWT contents.

The JWT token was decoded using JWT.io.

The token revealed several sensitive user attributes.

Figure 6 – JWT Token Decoding

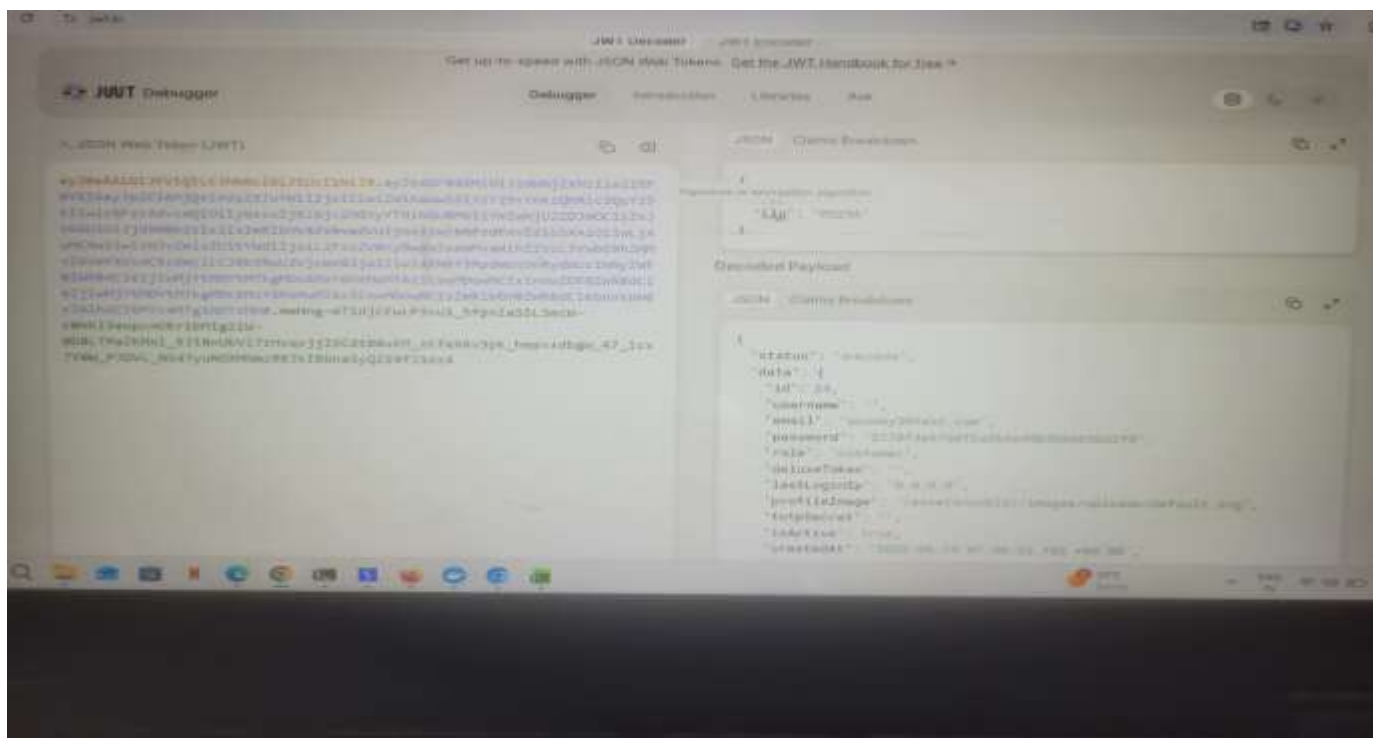
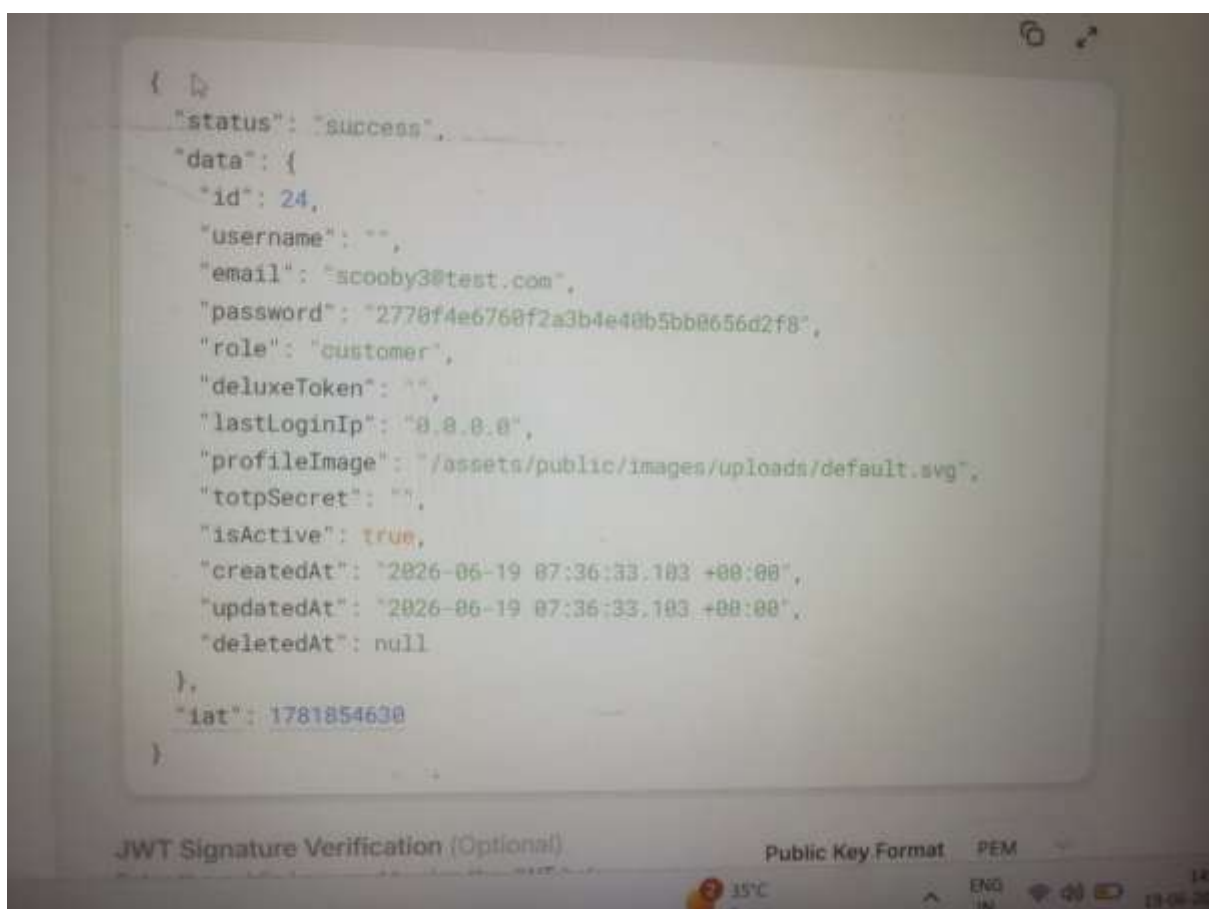


Figure 7 – JWT Payload Containing Sensitive User Data



Observation -

JWT contained:

- User ID
- Email Address
- Password Hash
- Role Information
- Internal Attributes

Risk

Sensitive data should not be stored inside JWT payloads.

➤ Finding 1

Broken Object Level Authorization (BOLA)

Severity: High

OWASP API Top 10:

API1:2023 Broken Object Level Authorization

Description

The API allowed direct access to resources by changing object identifiers.

By modifying basket IDs, data belonging to other users became accessible.

Steps to Reproduce

1. Intercept request
 - GET /rest/basket/6
2. Change basket ID
 - GET /rest/basket/4

3. Observe another user's basket information

Figure 8 – GET /rest/basket/6

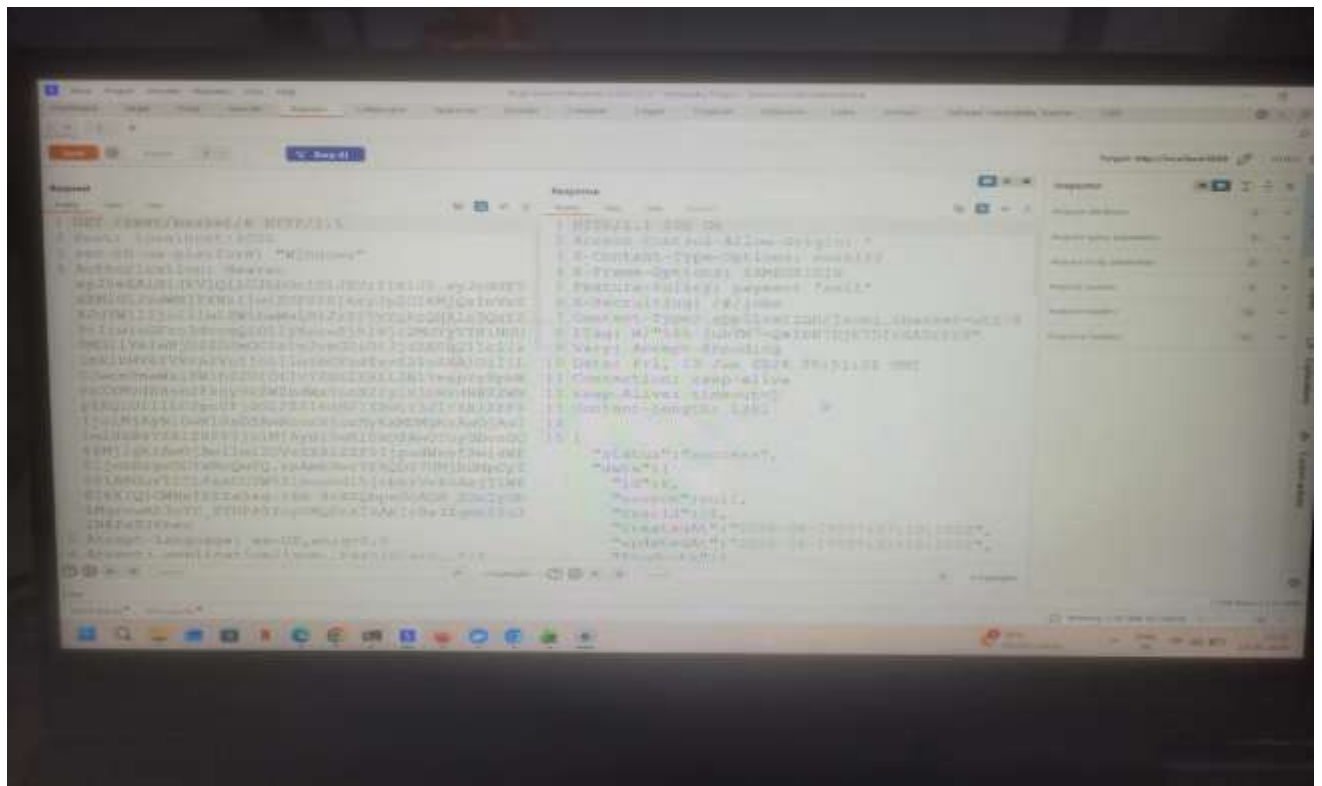


Figure 9 – Accessing Basket ID 4

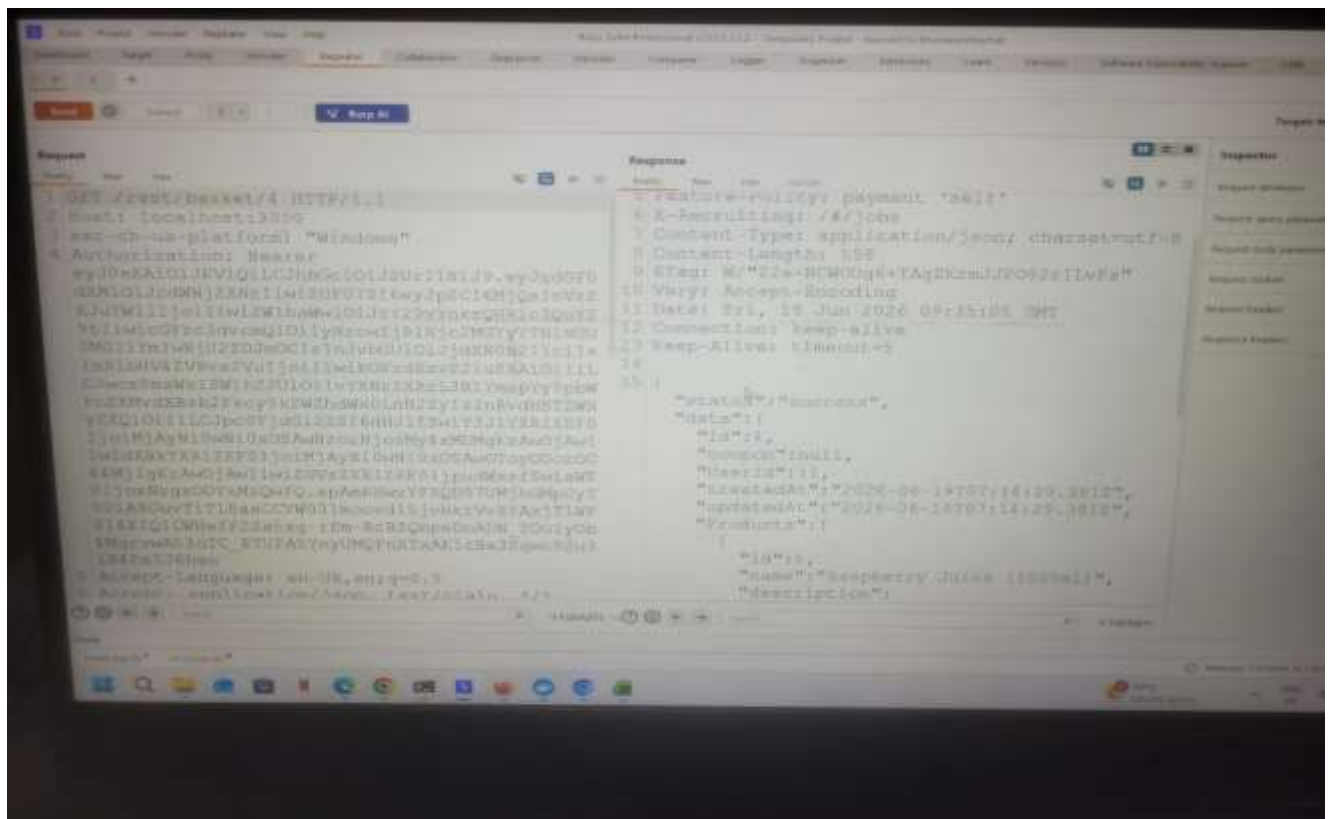
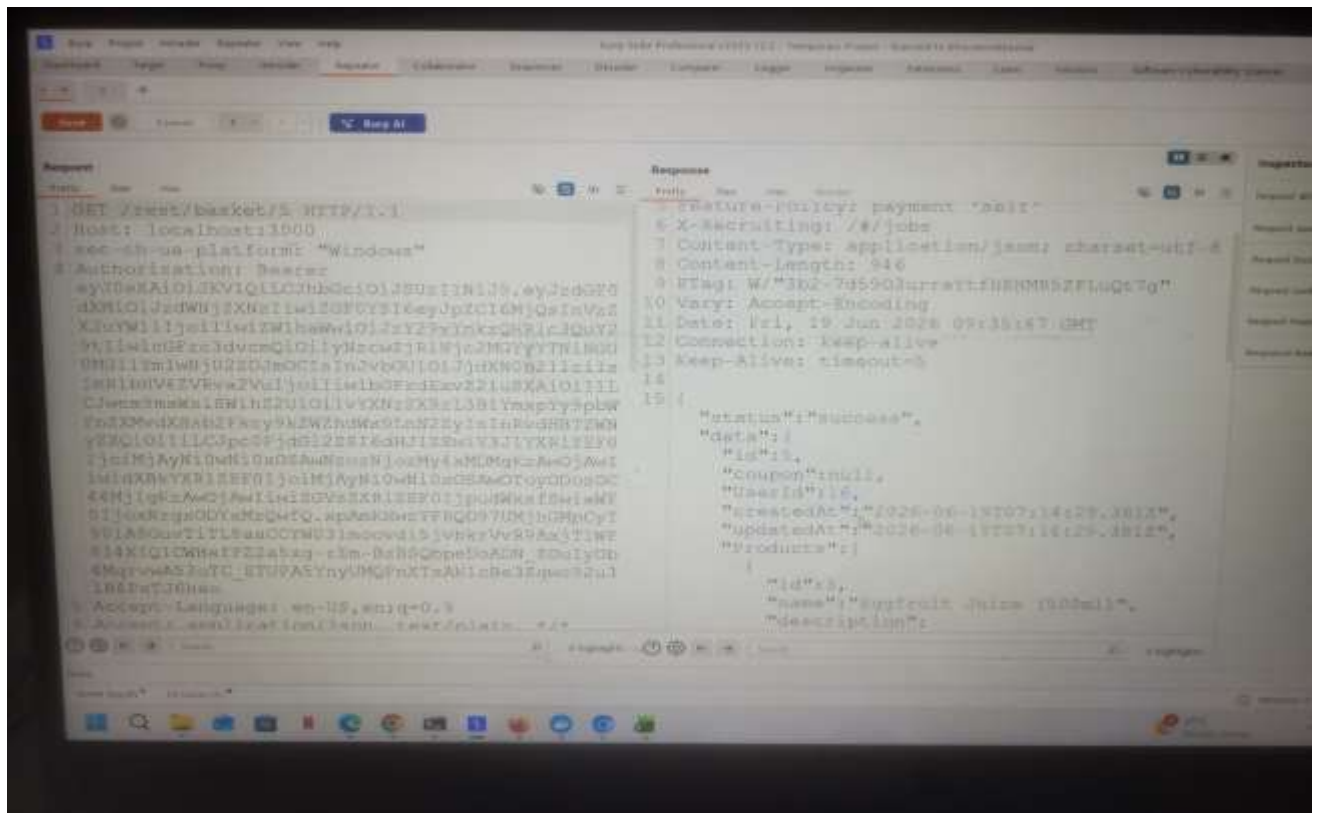


Figure 10 – Accessing Basket ID 5



Impact -

An attacker can:

- Access other users' shopping baskets
- View purchase information
- Enumerate customer data

Recommendation

Implement server-side authorization checks validating resource ownership.

➤ Finding 2

Excessive Data Exposure

Severity: Medium

OWASP API Top 10:

API3:2023 Broken Object Property Level Authorization

Description

The API exposes internal user attributes that are unnecessary for clients.

The endpoint supports querying individual user properties.

Evidence

Figure 11 – Email Address Exposure

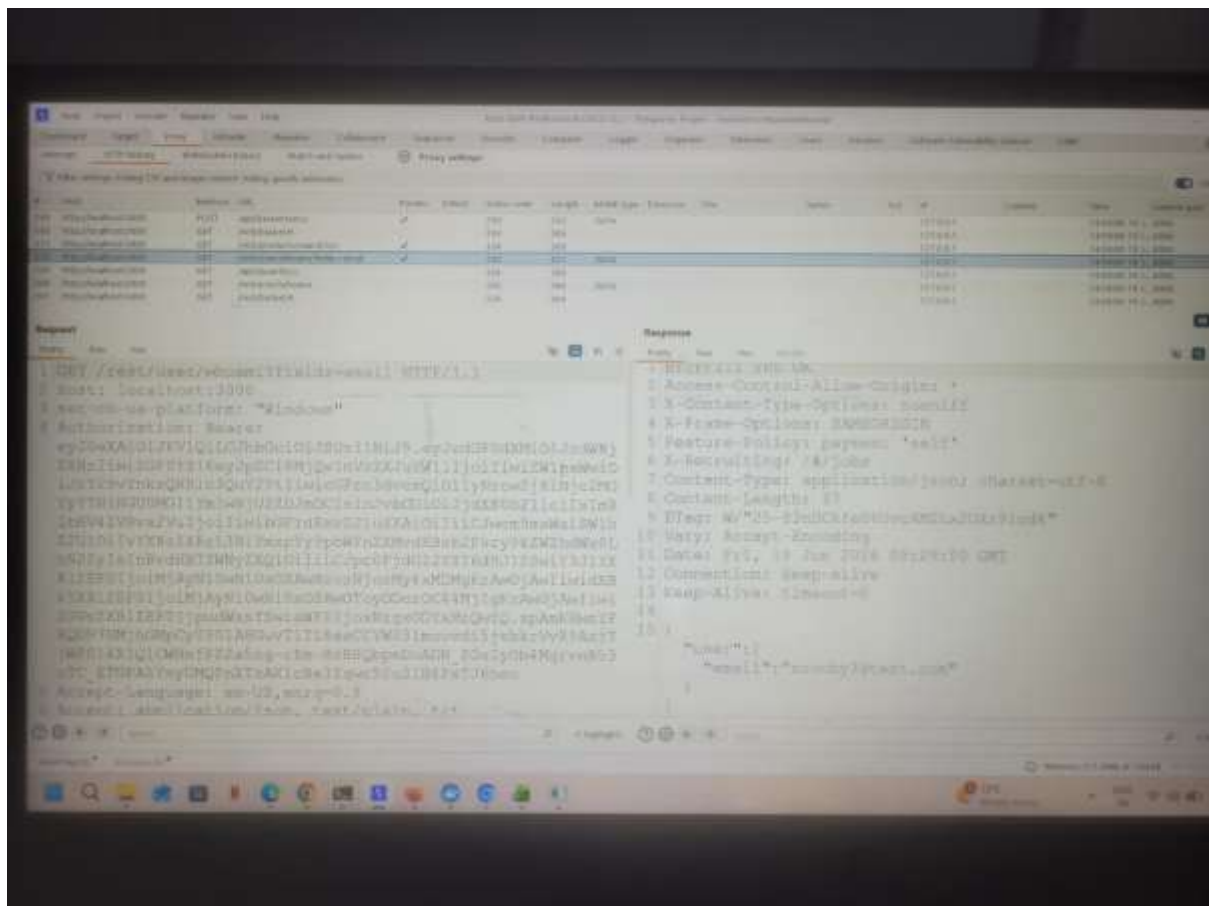


Figure 12 – User ID Exposure

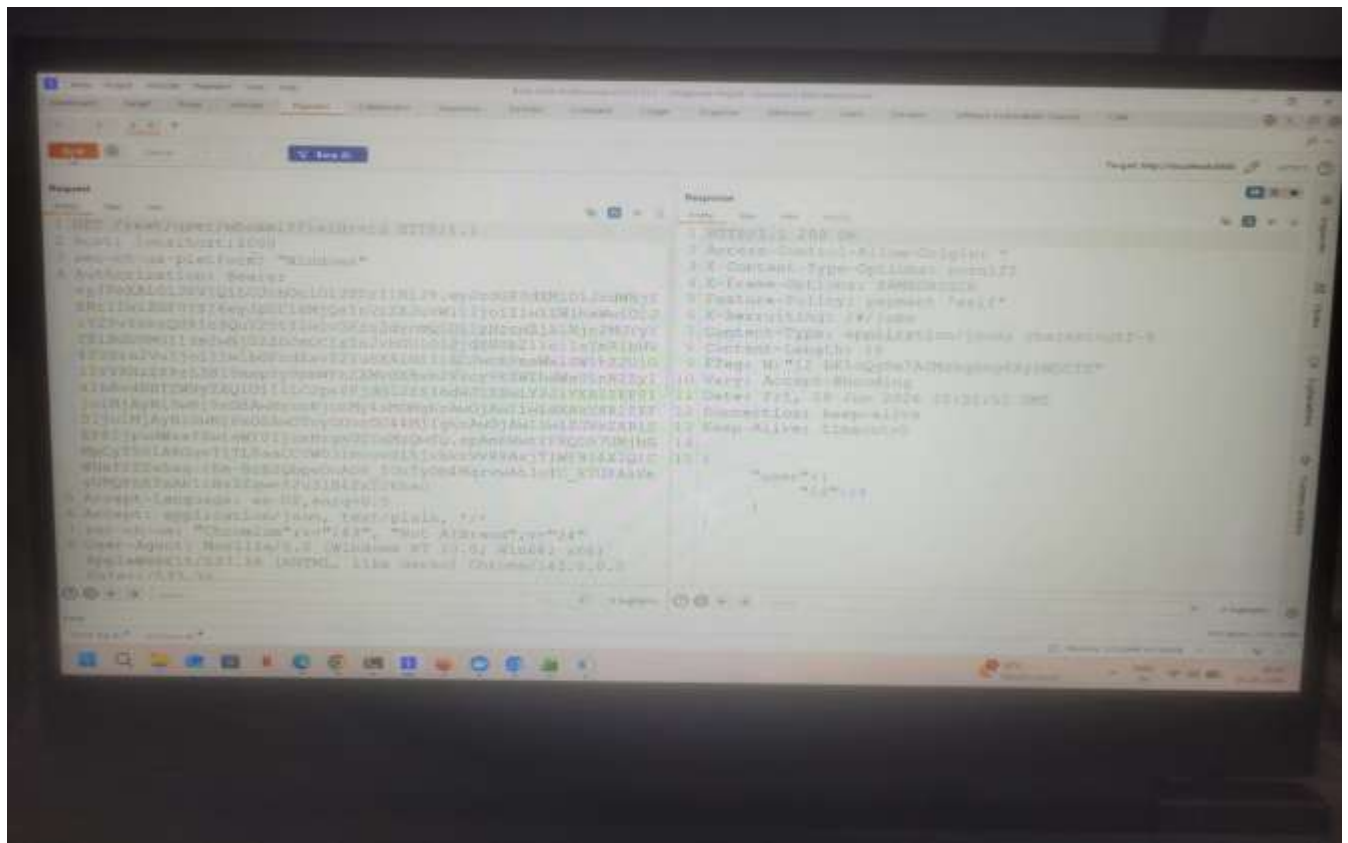


Figure 13 – Role Information Exposure

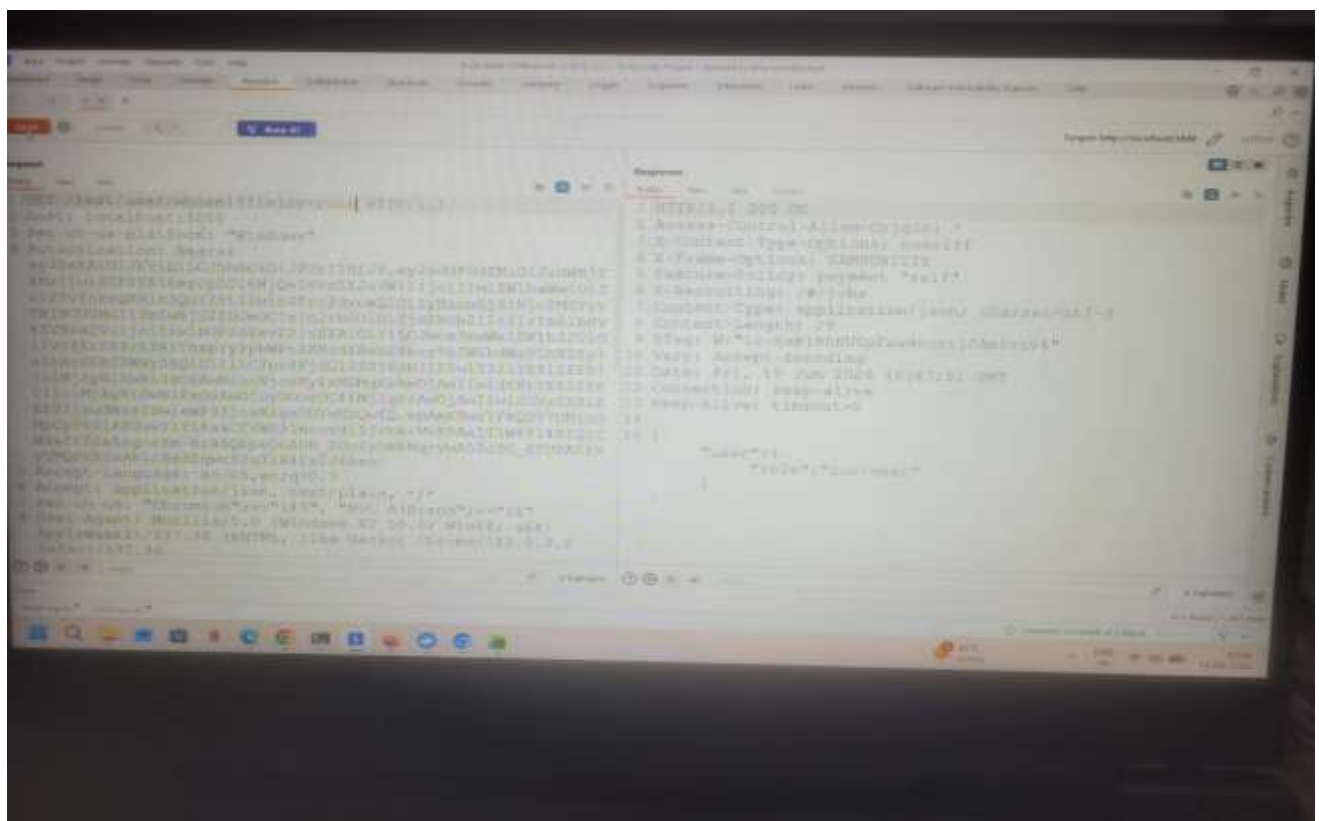


Figure 14 – Password Hash Exposure

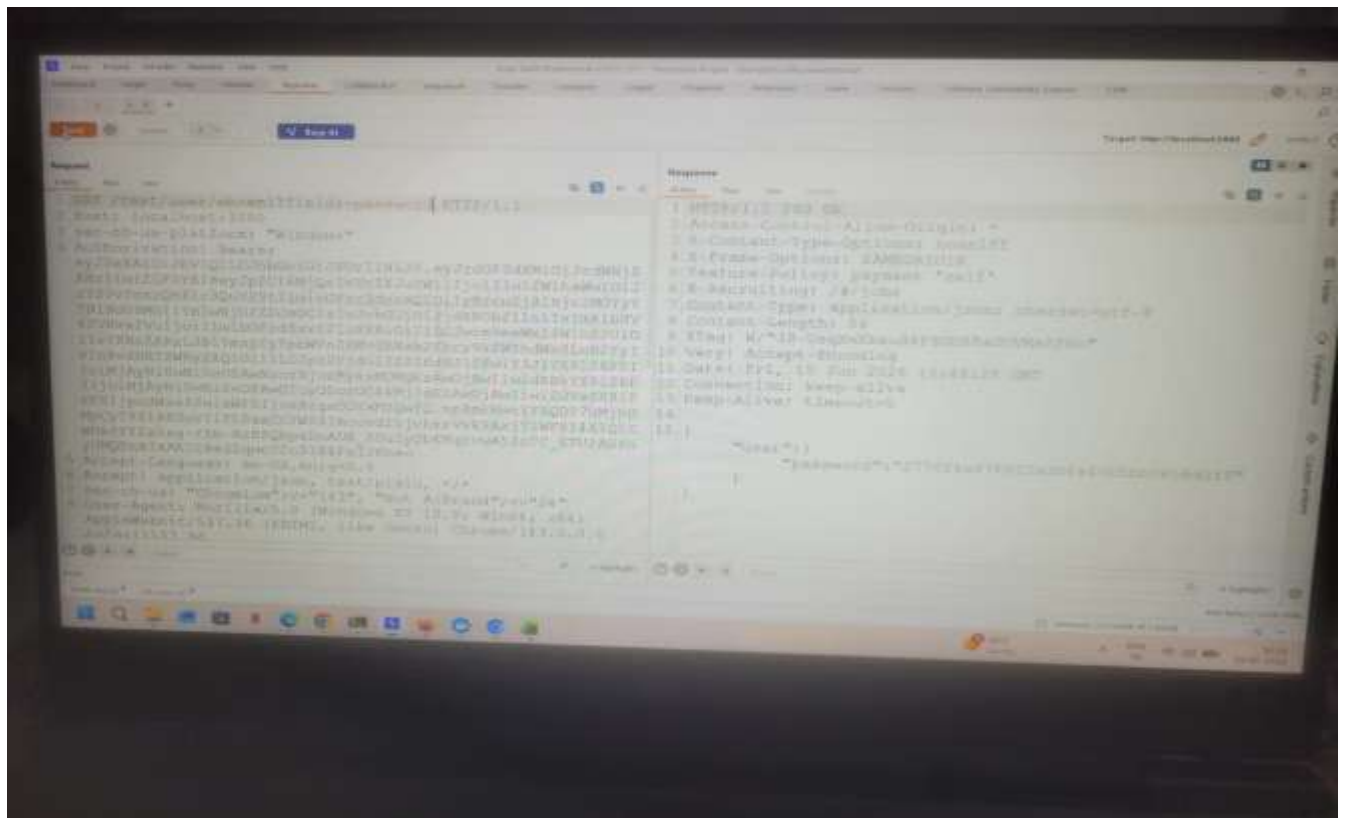
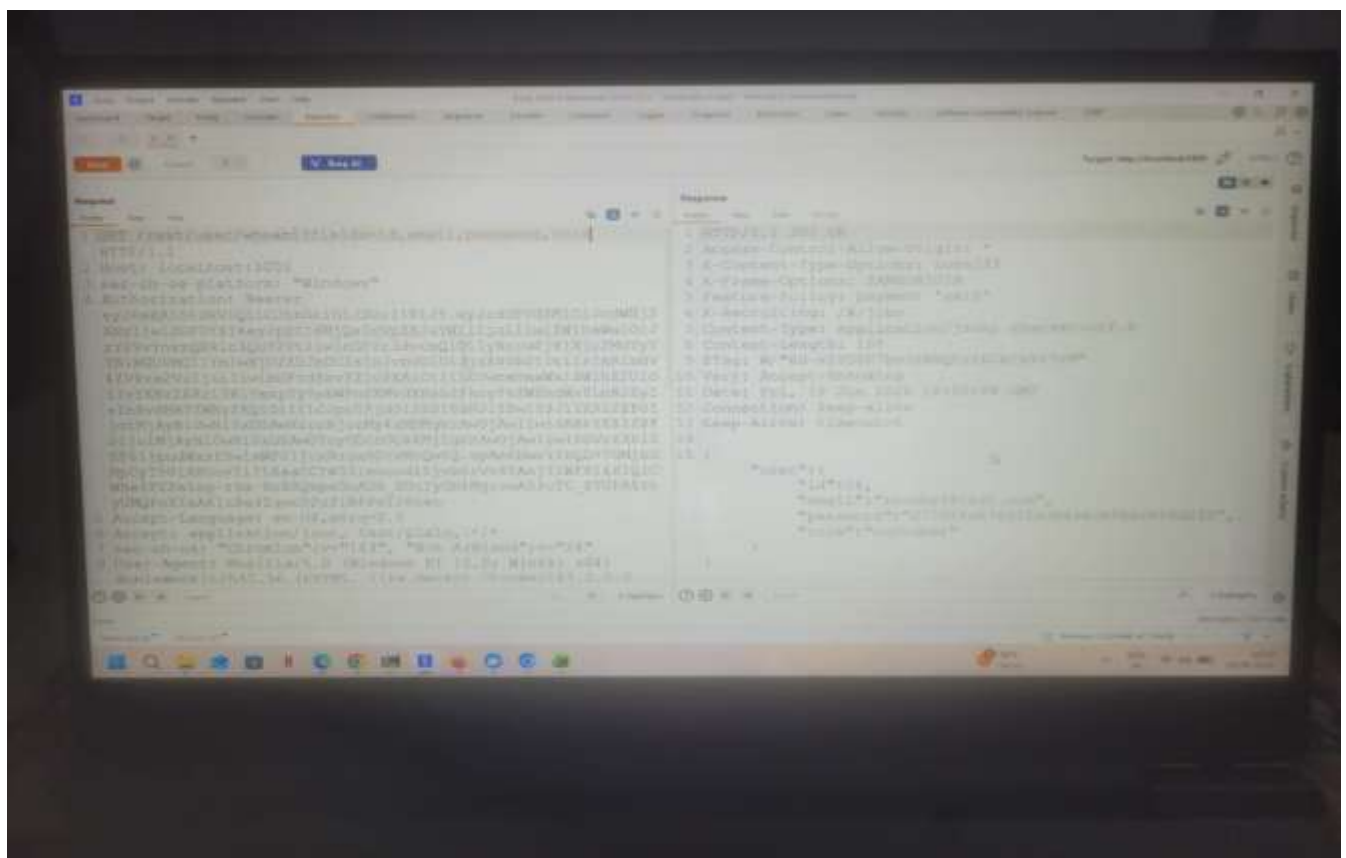


Figure 15 – Multiple Sensitive Attributes Returned



Impact -

Attackers can gather:

- Email addresses
- Internal identifiers
- User roles
- Password hashes

which may facilitate privilege escalation and credential attacks.

Recommendation

Return only required fields to the client.

➤ Finding 3

Broken Authentication

Severity: High

OWASP API Top 10:

API2:2023 Broken Authentication

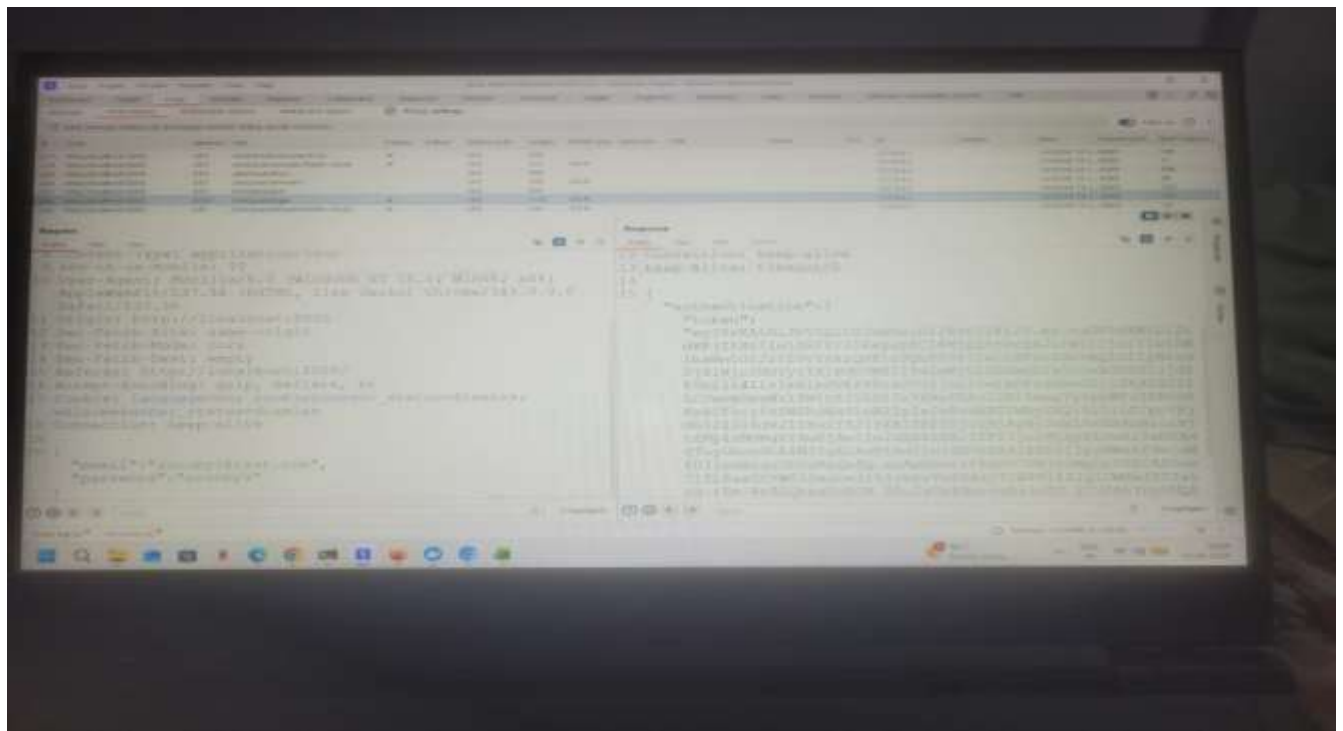
Description

The API accepts malformed authentication tokens and still processes requests.

This indicates insufficient JWT validation.

Step 1 – Valid JWT

Figure 16 – Valid Authentication Token



Step 2 – Modified Token

Figure 17 – Request with Valid JWT Producing Successful Response

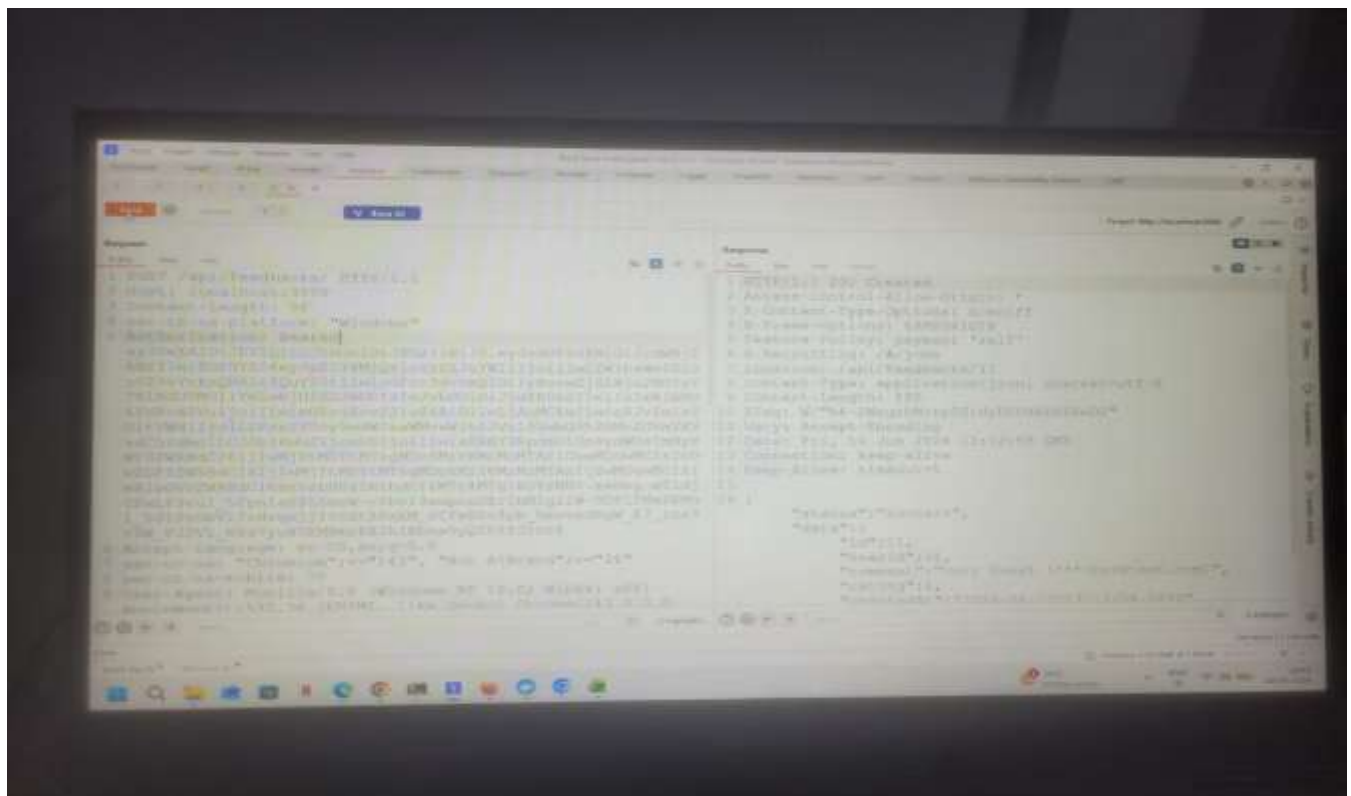
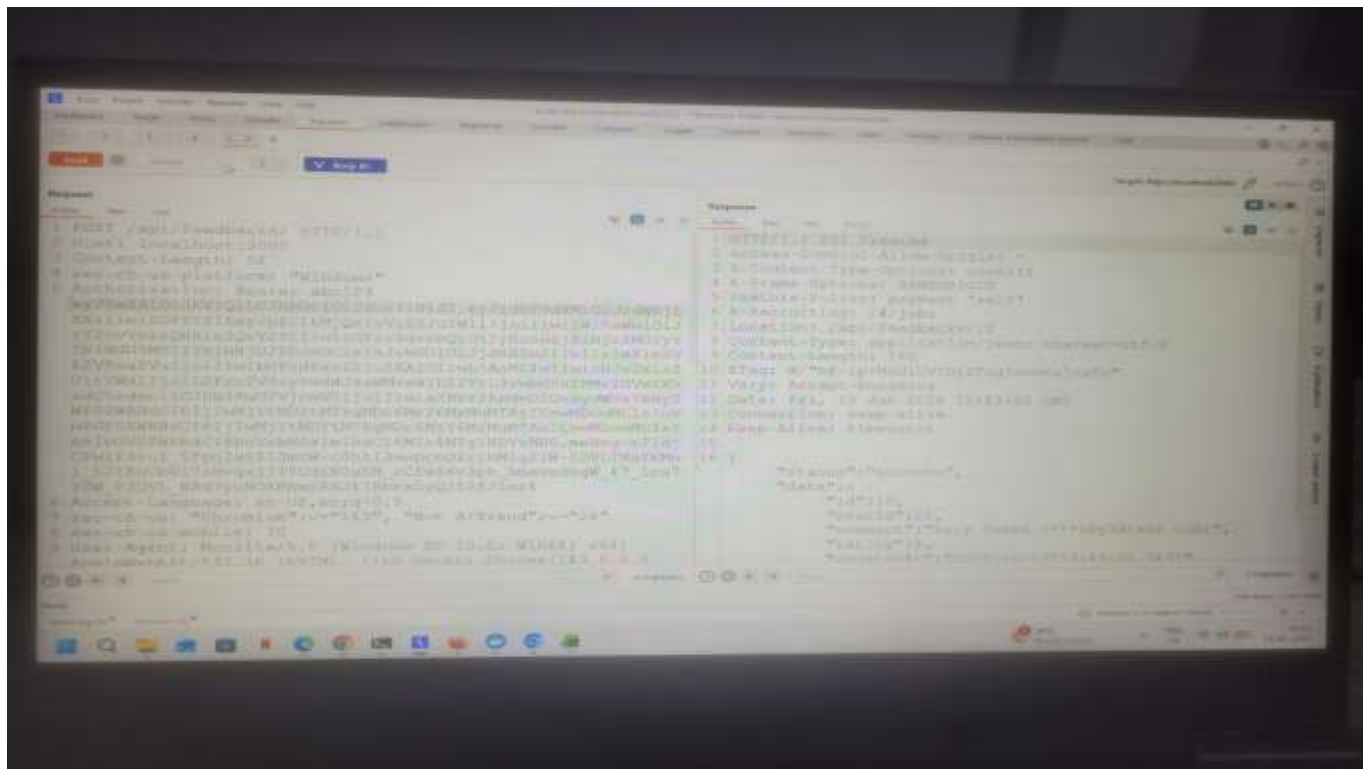
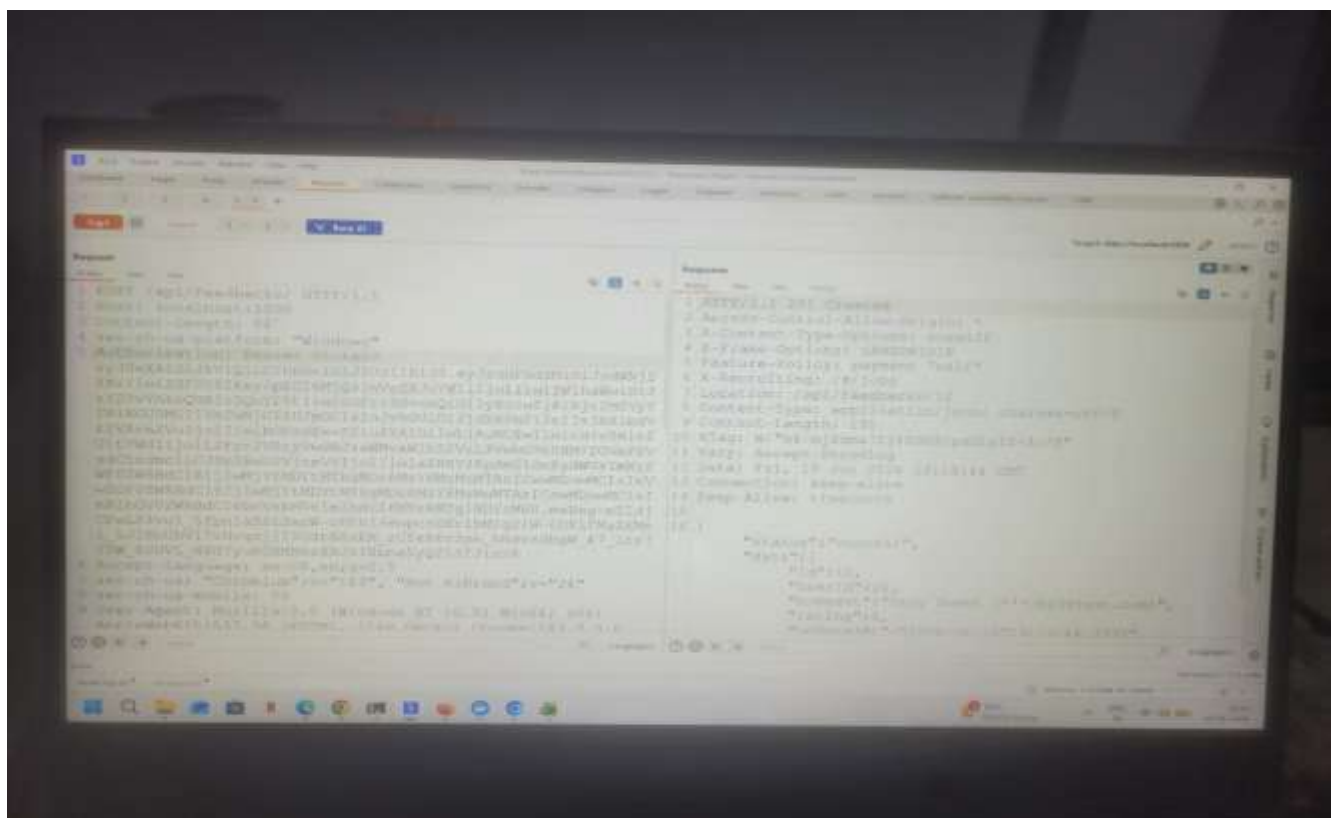


Figure 18 – Authentication Header Modified to 'Bearer abc123'



Step 3 – Invalid Token Accepted

Figure 19 – Invalid Token Still Accepted by Server



Observation

Expected:

- 401 Unauthorized

Actual:

- 201 Created

Request succeeded despite invalid authentication token.

Impact -

An attacker may:

- Bypass authentication controls
- Perform unauthorized actions
- Access protected API functionality

Recommendation -

Implement strict JWT validation:

- Verify signature
- Verify expiration
- Reject malformed tokens
- Reject missing tokens

4. Risk Summary

Vulnerability	Severity
1. Broken Authentication	High
2. Broken Object Level Authorization	High
3. Excessive Data Exposure	Medium
4. Sensitive Information Exposure	Medium

5. Conclusion

The API security assessment identified several vulnerabilities affecting authentication, authorization, and data exposure controls.

The most critical findings were:

1. Broken Authentication
2. Broken Object Level Authorization (BOLA)

These weaknesses could allow unauthorized access to user data and application functionality. Immediate remediation is recommended before deployment in production environments.

